

Domain-Model-v1.0



Όνομα Έργου: SpecFlow

Κωδικός: Domain-Model

Έκδοση: v1.0

Μέλη Ομάδας

Τα μέλη της ομάδας μας είναι:

- Παναγιώτης Αδριανός, AM: 1097451.
- Παρασκευάς Παπαγιαννόπουλος, AM: 1104810.
- Ιωάννης Αντωνόπουλος, AM: 1065023.
- Νικόλαος Αντωνόπουλος, AM: 1098210.
- Δημήτριος Κούκος, AM: 1080483.

Κατανομή Ρόλων στο παρόν τεχνικό κείμενο

Η κατανομή των ρόλων είναι η ακόλουθη:

- Παρασκευάς Παπαγιαννόπουλος: Contributor, Peer Reviewer
- Παναγιώτης Αδριανός: Contributor, Peer Reviewer
- Ιωάννης Αντωνόπουλος: Contributor, Peer Reviewer
- Δημήτριος Κούκος: Contributor, Peer Reviewer
- Νικόλαος Αντωνόπουλος: Contributor, Peer Reviewer

Στο παρόν τεχνικό κείμενο όλα τα μέλη της ομάδας συνεισέφεραν εξίσου στην περιγραφή των κλάσεων του έργου. Ακόμα σας παραθέτουμε και link για το repository της ομάδας μας στο [GitHub](#).

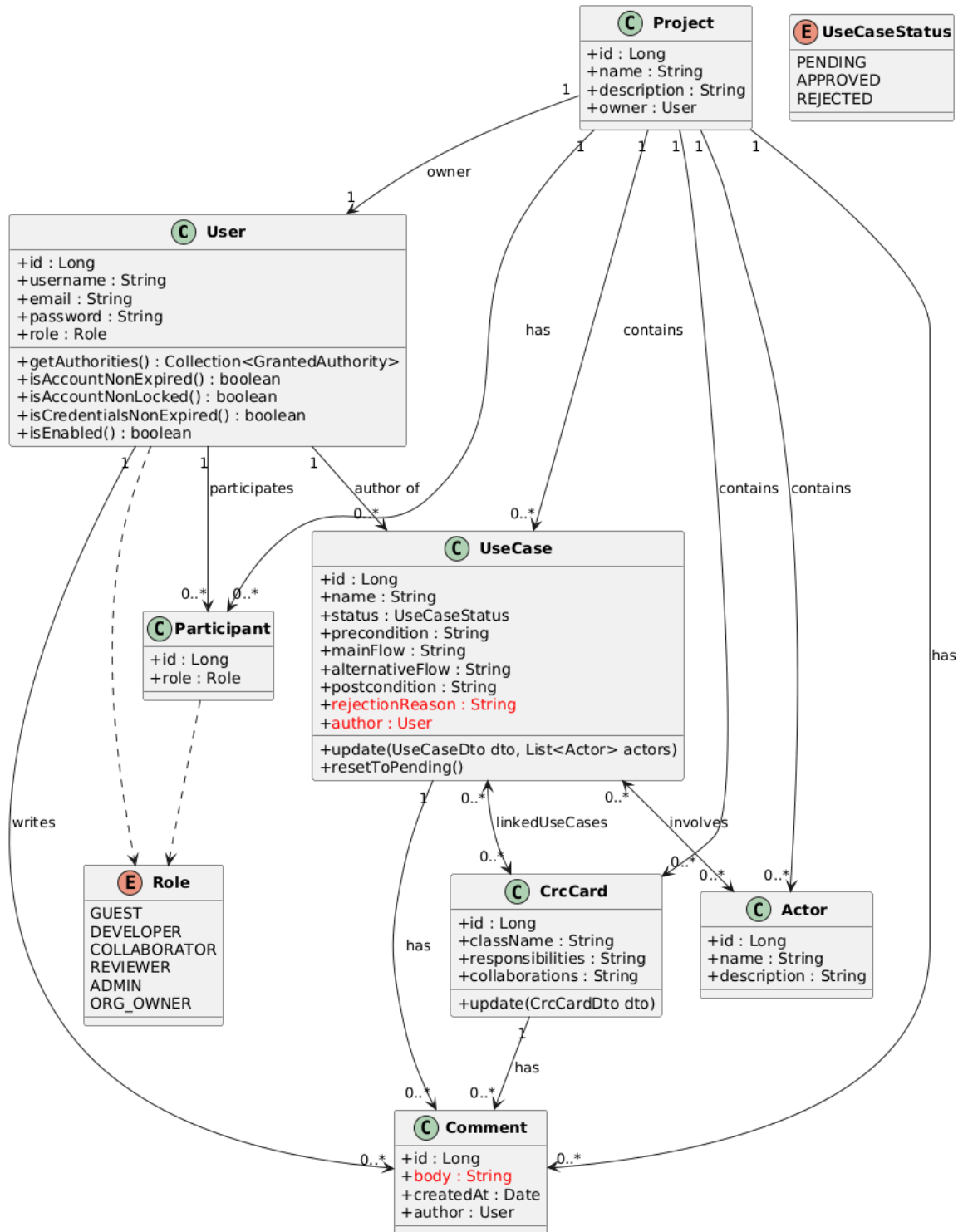
Αλλαγές και Βελτιώσεις

Η τελική έκδοση 1.0 (v1.0) του εγγράφου ενσωματώνει τις **αλλαγές και βελτιώσεις** που προέκυψαν αναγκαστικά κατά τη συγγραφή του κώδικα και των test cases, μετατρέποντας το αρχικό θεωρητικό σχήμα σε μια ώριμη, ασφαλή και επεκτάσιμη αρχιτεκτονική παραγωγής. Συγκεκριμένα, εισήχθησαν τα **DTOs** για να απομονώσουν τις οντότητες της βάσης από τις φόρμες και να προσφέρουν άμεσο validation στην είσοδο, ενώ δημιουργήθηκε ένα οργανωμένο **Exceptions Layer** ώστε κάθε επιχειρησιακό σφάλμα να αποκτά σαφές νόημα και να αντιμετωπίζεται κεντρικά από τον GlobalExceptionHandler, εξαλείφοντας τα τυχαία κρashaρίσματα.

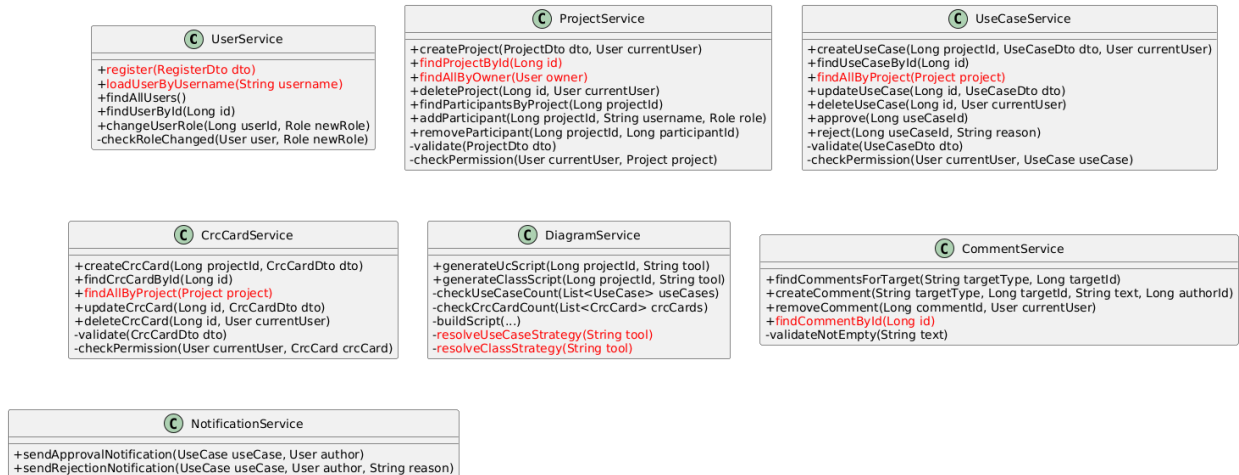
Παράλληλα, το **Security Layer** αναμορφώθηκε πλήρως για να προσφέρει πραγματική session-based ασφάλεια, κρυπτογράφηση κωδικών με BCrypt και προστασία CSRF, θωρακίζοντας την εφαρμογή βάσει ρόλων. Τέλος, η παραγωγή των διαγραμμάτων απομονώθηκε μέσω του **Strategy Pattern**, επιτρέποντας στο σύστημα να υποστηρίζει PlantUML και Nomnoml (και μελλοντικά οποιοδήποτε νέο εργαλείο) με απλή προσθήκη κλάσεων, χωρίς να επηρεάζεται η υπάρχουσα επιχειρησιακή λογική. Με αυτές τις διορθώσεις, η v1.0 εξασφαλίζει τον απόλυτο διαχωρισμό ευθυνών και την καθαρότητα του κώδικα.

Διάγραμμα Domain Model

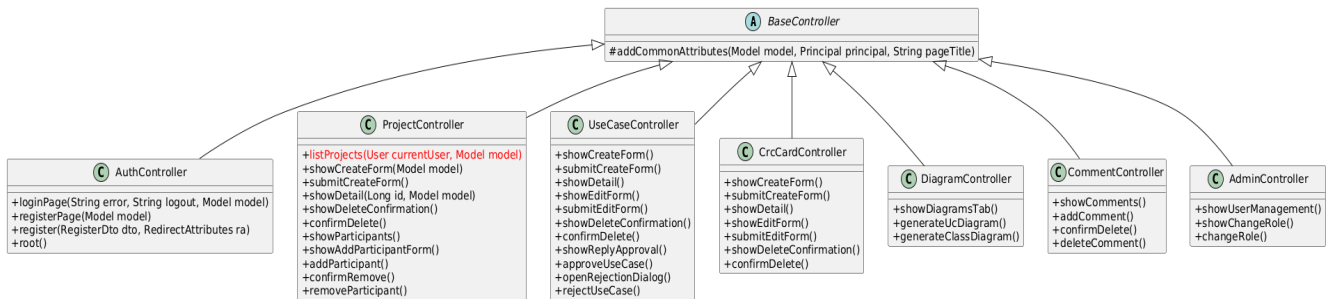
Στα παρακάτω σχήματα παρουσιάζουμε το διάγραμμα κλάσεων του Domain Model χωρισμένο σε επιμέρους διαγράμματα για μεγαλύτερη καθαρότητα (Entities, Services, Controllers, Repositories, **DTOs**, **Strategies**, **Exceptions**, **Security**). Στην παρούσα έκδοση του συγκεκριμένου τεχνικού κειμένου, παρουσιάζουμε τις κλάσεις, τις συσχετίσεις μεταξύ τους, συναρτήσεις και πεδία που περιέχουν τα οποία προέκυψαν από την **εκπόνηση του Κώδικα και των Test Cases**. Τέλος φαίνεται κι η επικοινωνία μεταξύ των επιπέδων ανα ζεύγη. Με κόκκινα είναι όλες οι σημαντικές αλλαγές **αλλά και η συνολική αρχιτεκτονική**.



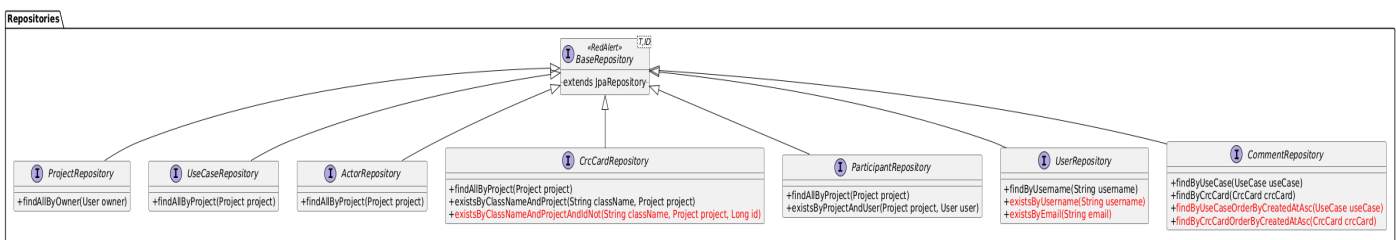
Εικόνα 1: Διάγραμμα Κλάσεων



Εικόνα 2: Services (Business Logic Layer)

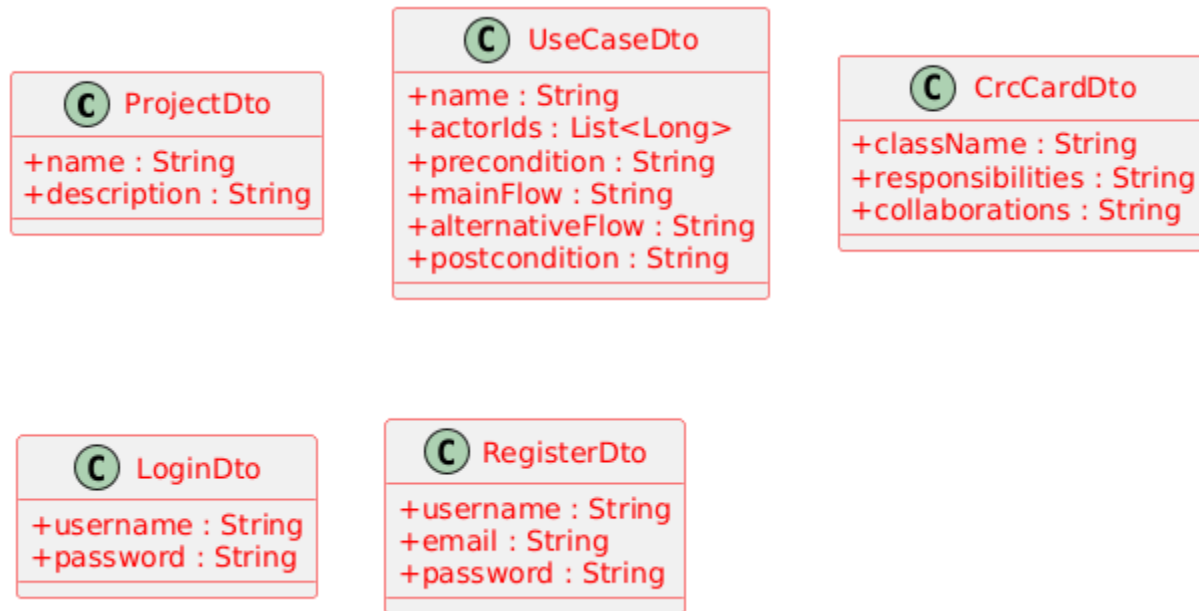


Εικόνα 3: Controllers (UI / HTTP Layer)

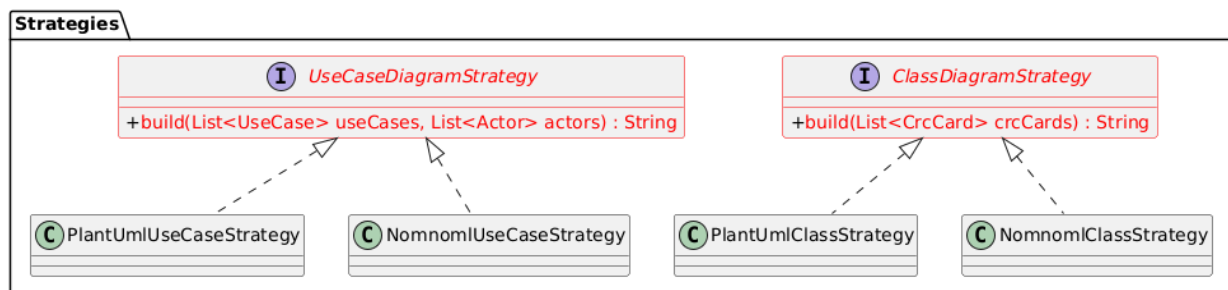


Εικόνα 4: Repository

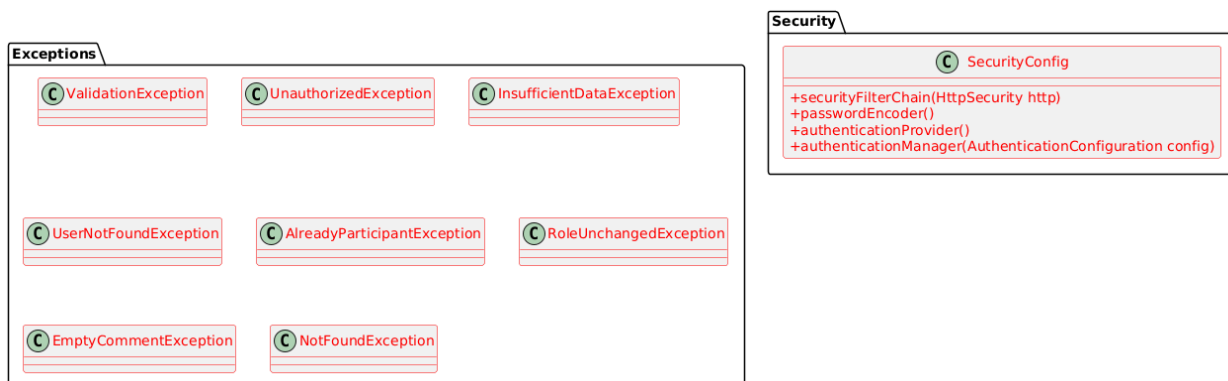
Δέχεται τα HTTP Requests. Δεν γνωρίζει τίποτα για τη βάση δεδομένων. Καλεί αποκλειστικά το **Service Layer**.



Εικόνα 5: DTOs

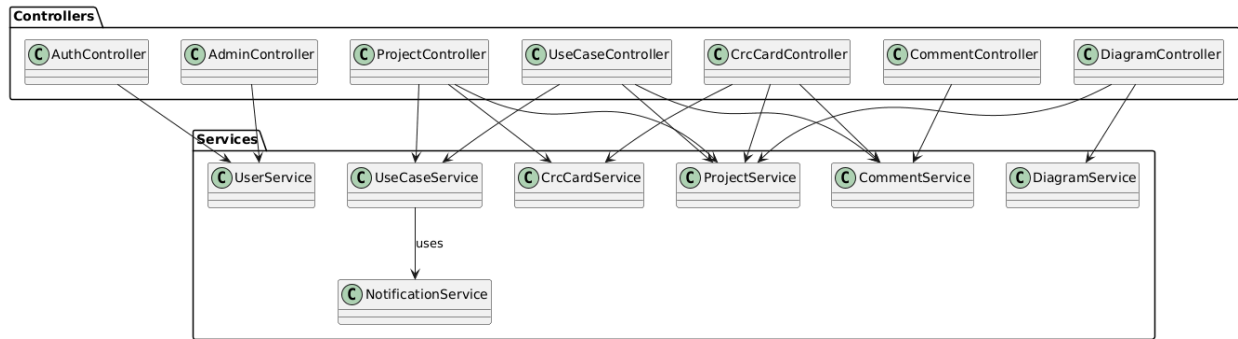


Εικόνα 6: Strategies

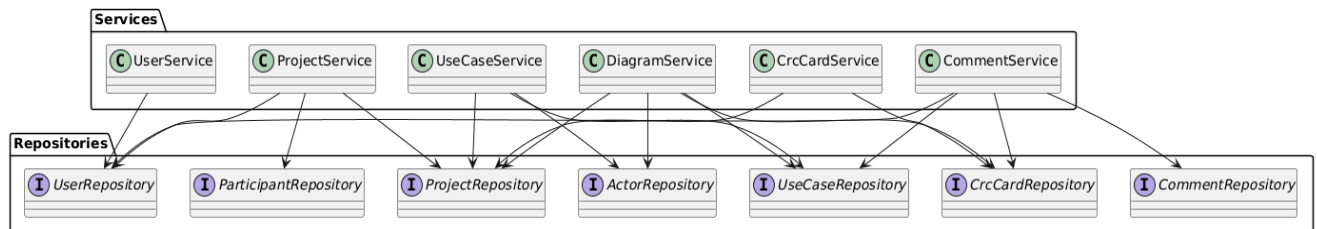


Εικόνα 7: Exceptions & Security

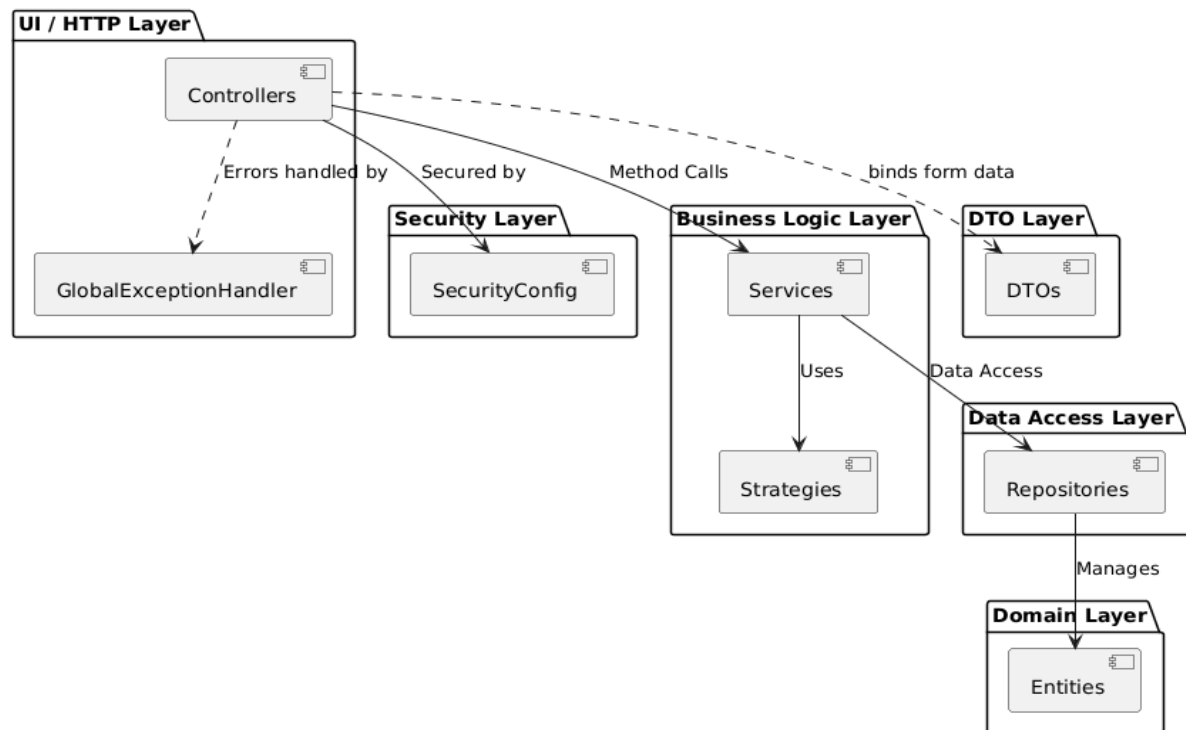
Διασυνδέσεις Επιπέδων - Layers



Εικόνα 8: Controller – Service (communication)



Εικόνα 9: Services-Repositories



Εικόνα 10: Συνολικό Διάγραμμα Αρχιτεκτονικής

Περιγραφή κλάσεων

Στην παρακάτω περιγραφή των κλάσεων στοχεύουμε να φανούν οι βασικές υποψήφιες κλάσεις. Καθώς θα προχωρήσει η ανάλυση και η υλοποίηση, ενδέχεται να προστεθούν ή να αφαιρεθούν κλάσεις ή να γίνουν πιο εξειδικευμένες οι σχέσεις.

A. ENTITIES

- **User:** Οντότητα που αντιστοιχεί σε εγγεγραμμένο άτομο που χρησιμοποιεί την εφαρμογή. Ανάλογα με τον ρόλο του, μπορεί να είναι Προγραμματιστής (**Developer**), Συνεργάτης (**Collaborator**), Αξιολογητής (**Reviewer**), Ιδιοκτήτης Οργανισμού (**Organization Owner**) ή Διαχειριστής (**Admin**). Κάθε **User** μπορεί να ιδιοκτητεύει Έργα, να συμμετέχει σε άλλα Έργα ως προσκεκλημένος, και να αφήνει Σχόλια. Συσχετίζεται (1-1) με το Role και (1-N) με ProjectMembership και Comment.
- **Role:** Οντότητα που αναπαριστά τους διάφορους ρόλους ενός χρήστη που μπορεί να είναι Προγραμματιστής (**Developer**), Συνεργάτης (**Collaborator**), Αξιολογητής (**Reviewer**), Ιδιοκτήτης Οργανισμού (**Organization Owner**) ή Διαχειριστής (**Admin**).
- **Project:** Οντότητα που αντιστοιχεί στο βασικό αντικείμενο εργασίας στο SpecFlow. Αντιστοιχεί σε ένα έργο ανάπτυξης λογισμικού. Ανήκει σε έναν Χρήστη-ιδιοκτήτη και περιέχει Περιπτώσεις Χρήσης, Χειριστές και Κάρτες CRC. Ο Ιδιοκτήτης Οργανισμού έχει πρόσβαση σε όλα τα Έργα του οργανισμού. Συσχετίζεται (1-N) με UseCase, Actor, CrcCard, ProjectMembership και Comment.
- **Participant:** Association Entity που αντιπροσωπεύει τη συμμετοχή ενός χρήστη σε ένα Project με συγκεκριμένο ρόλο (π.χ. Collaborator, Reviewer). Αντικαθιστά την παλαιότερη έννοια του ProjectMembership. Συσχετίζεται (N-1) με User και Project.
- **UseCase:** Κλάση που αντιστοιχεί σε μια απαίτηση χρήσης που περιγράφει μία λειτουργία του συστήματος που αναπτύσσεται. Αναφέρει τους **Actors** που εμπλέκονται, την προϋπόθεση εισόδου, τη βασική ροή, εναλλακτικές ροές και την κατάσταση εξόδου. Συνδέεται με Κάρτες CRC που την υποστηρίζουν. Φέρει κατάσταση έγκρισης (PENDING, APPROVED, REJECTED) που ορίζεται από τον Ιδιοκτήτη Οργανισμού. Συσχετίζεται (M-M) με Actor και CrcCard, και (1-N) με Comment. Πλέον υποστηρίζει τον πλήρη κύκλο ενημέρωσης των στοιχείων της και την επαναφορά της σε κατάσταση αναμονής, καθώς και την ακριβή καταγραφή του λόγου απόρριψης.
- **Actor:** οντότητα που εμπλέκεται σε μία ή περισσότερες Περιπτώσεις Χρήσης ενός Έργου. Ορίζεται από τον ιδιοκτήτη του **Project**.
- **CRCCard:** Οντότητα που αναπαριστά την κάρτα Κλάσης-Αρμοδιοτήτων-Συνεργασιών (Class-Responsibility-Collaboration). Αναπαριστά μία κλάση του συστήματος που σχεδιάζεται, αναφέροντας τι κάνει (αρμοδιότητες) και με ποιες άλλες κλάσεις συνεργάζεται. Συσχετίζεται (M-M) με UseCase και (1-N) με Comment. Επιτρέπει την πλήρη ενημέρωση των πεδίων της μέσω εξειδικευμένων διαδικασιών.

- **ProjectMembership:** Αντιπροσωπεύει τη συμμετοχή ενός χρήστη σε ένα Project με συγκεκριμένο ρόλο (Collaborator, Reviewer). Υλοποιεί την περίπτωση όταν ένας ιδιοκτήτης μοιράζεται το project του. Συσχετίζεται (N-1) με User και Project.
- **Comment:** Οντότητα που αναπαριστά την παρατήρηση ή ερώτηση που αφήνει ένας **User** πάνω σε μια ΠερίπτωσηΧρήσης ή ΚάρταCRC. Χρησιμεύει για feedback, επικοινωνία μεταξύ μελών της ομάδας, αλλά και για αιτιολόγηση απόρριψης απαίτησης από τον Ιδιοκτήτη Οργανισμού. Φέρει το κείμενο του σχολίου (body) και την ώρα δημιουργίας (createdAt). Συσχετίζεται (N-1) με τον User.

B. SERVICES

- **UserService:** Κλάση που παρέχει λειτουργίες για εγγραφή (signup), σύνδεση (login) και έξοδο (logout) των οντοτήτων **User**. Πλέον, συγκεντρώνει την κεντρική λογική διαχείρισης των χρηστών και εφαρμόζει με ασφάλεια τους κανόνες του συστήματος όταν ο Διαχειριστής αποφασίζει να τροποποιήσει τα προνόμια ή τον ρόλο ενός εγγεγραμμένου μέλους.
- **ProjectService:** Κλάση που παρέχει λειτουργίες για την εμφάνιση όλων των έργων ενός συνδεδεμένου **User** και για την δημιουργία ή την διαγραφή των αντικειμένων **Project**. Επιπλέον, διασφαλίζει την ακεραιότητα προσθέτοντας ελέγχους πρόσβασης και υλοποιεί την πολύπλοκη λογική του διαμοιρασμού, επιβεβαιώνοντας αν το Username που προσκαλείται υπάρχει και προστατεύοντας από διπλότυπες συμμετοχές.
- **UseCaseService:** Κλάση που παρέχει λειτουργίες για την δημιουργία ενός καινούργιου αντικειμένου **UseCase** ή την ενημέρωση ενός υπάρχοντος. Ακόμα, παρέχει και λειτουργίες διαγραφής ενός **UseCase**, αλλά και εμφάνισης όλων των περιπτώσεων χρήσης ενός **Project**. Ελέγχει την εγκυρότητα των πεδίων (π.χ. αναγκαστική εισαγωγή Name και Actor) και αναλαμβάνει την αλλαγή της κατάστασης μίας απαίτησης, όπως την επαναφορά της σε PENDING ύστερα από επεξεργασία ή την κατοχύρωση της τελικής απόφασης έγκρισης ή απόρριψης.
- **CRCCardService:** Κλάση που παρέχει λειτουργίες για την δημιουργία ενός καινούργιου αντικειμένου **CRCCard** ή την ενημέρωση ενός υπάρχοντος. Ακόμα, παρέχει και λειτουργίες διαγραφής ενός **CRCCard**, αλλά και εμφάνισης όλων των καρτών “Κλάσης-Αρμοδιοτήτων-Συνεργασιών” ενός **Project**. Πριν την αποθήκευση, επικοινωνεί με τη βάση για να διασφαλίσει ότι δεν υπάρχει άλλη κάρτα με το ίδιο ακριβώς Class Name στο ίδιο έργο, προστατεύοντας τη συνέπεια του συστήματος.
- **DiagramService:** Κλάση που παρέχει λειτουργίες για την δημιουργία ενός διαγράμματος “Περιπτώσεων Χρήσεις” ή ενός διαγράμματος “Κλάσεων” βασισμένη στα δεδομένα του έργου. Υλοποιεί τη λογική δυναμικής μετατροπής των δεδομένων του έργου σε κώδικα UML, αξιοποιώντας ένα επεκτάσιμο μοντέλο που αναθέτει τη δουλειά στις κατάλληλες εσωτερικές διαδικασίες ανάλογα με την επιλογή του χρήστη (π.χ. PlantUML, Nomnoml).
- **CommentService:** Κλάση που παρέχει λειτουργίες αποθήκευσης σχολίου το οποίο έχει γίνει σε μια συγκεκριμένη περίπτωση χρήσης από κάποιον **Collaborator** ή **Reviewer**.

Ελέγχει επιχειρησιακά τα δεδομένα της συζήτησης, διασφαλίζοντας ότι τα σχόλια δεν μπορούν να καταχωρηθούν κενά και επιβεβαιώνοντας ότι η εντολή διαγραφής προέρχεται αυστηρά από τον αρχικό συντάκτη του σχολίου.

- **NotificationService:** Μια νέα επιχειρησιακή λειτουργία που παρεμβαίνει ως ασύγχρονος παρατηρητής. Όταν κατοχυρωθεί η έγκριση ή η απόρριψη μιας απαίτησης, το συγκεκριμένο service αναλαμβάνει τη σύνταξη και την αποστολή προσωποποιημένων ειδοποιήσεων προς τους συντάκτες, κρατώντας όλα τα μέλη ενημερωμένα.

C. CONTROLLERS

Το πακέτο ελεγκτών περιλαμβάνει έξι διαφορετικούς ελεγκτές που κληρονομούν από έναν abstract **BaseController** για επαναχρησιμοποίηση κώδικα (π.χ. κοινά attributes).

Είναι υπεύθυνο για την αλληλεπίδραση των χρηστών με το σύστημα, διαχειριζόμενο τα HTTP requests και την επιστροφή των κατάλληλων Views, συνδεδεμένο άμεσα με το Service Layer:

- Ο **AuthController** είναι υπεύθυνος για την εγγραφή, την ταυτοποίηση και την είσοδο/έξοδο των χρηστών.
- Ο **ProjectController:** Υπεύθυνος για την αλληλεπίδραση των χρηστών με τα Έργα, αναλαμβάνοντας λειτουργίες προβολής, δημιουργίας και διαγραφής. Διαχειρίζεται τη δρομολόγηση των αιτημάτων για τη φόρμα δημιουργίας, την οριστική διαγραφή και την προβολή λεπτομερειών, ενώ αναλαμβάνει την επικοινωνία με τις όψεις για τη διαχείριση και πρόσκληση συμμετεχόντων.
- Ο **UseCaseController:** Υπεύθυνος για την αλληλεπίδραση με τις Περιπτώσεις Χρήσης. Αποτελεί τον βασικό ελεγκτή για τον κύκλο ζωής των απαιτήσεων, συντονίζοντας τις φόρμες δημιουργίας και επεξεργασίας, επιβλέποντας τη διαγραφή και δρομολογώντας τα κρίσιμα αιτήματα για την έγκριση ή την απόρριψη μιας απαίτησης από τον ιδιοκτήτη.
- Ο **CrcCardController:** Υπεύθυνος για την αλληλεπίδραση με τις Κάρτες CRC. Ελέγχει την εμφάνιση των φορμών για προσθήκη ή επεξεργασία, προβάλλει αναλυτικά τις Αρμοδιότητες και τις Συνεργασίες και χειρίζεται την επιβεβαίωση της διαγραφής τους.
- Ο **DiagramController :** Υπεύθυνος για τη δημιουργία και προβολή των διαγραμμάτων αρχιτεκτονικής. Συντονίζει την οπτικοποίηση, συλλέγει την προτίμηση του χρήστη ως προς το εργαλείο παραγωγής και επιστρέφει το τελικό παραγόμενο script προς αντιστοίχιση.
- Ο **CommentController:** Χειρίζεται την ανάρτηση και διαγραφή σχολίων από τους χρήστες. Εστιάζει αποκλειστικά στη διαχείριση της ροής των συζητήσεων, εμφανίζοντας τη φόρμα προσθήκης σχολίου κάτω από κάθε οντότητα και διασφαλίζοντας την ασφαλή ροή διαγραφής τους.
- **AdminController:** Ένας εξειδικευμένος ελεγκτής που δημιουργήθηκε για τις ανάγκες διαχείρισης του συστήματος. Αναλαμβάνει την παρουσίαση του πίνακα ελέγχου (Admin Panel) και δρομολογεί τα αιτήματα του διαχειριστή για την αλλαγή των καθολικών ρόλων οποιουδήποτε χρήστη.

D. REPOSITORIES

Η διαχείριση δεδομένων στηρίζεται πλέον σε ένα κοινό σημείο επέκτασης, την **BaseRepository<T, ID>**, την οποία κληρονομούν όλα τα ειδικά repositories. Το συγκεκριμένο interface επεκτείνει το JpaRepository του Spring Data JPA και είναι σημειωμένο με το annotation @NoRepositoryBean, ώστε να μην αναπαράχθει ως bean αλλά να αποτελέσει αμιγώς σημείο επέκτασης. Έτσι, οποιαδήποτε μελλοντική προσθήκη κοινής λειτουργικότητας (π.χ. soft-delete, auditing, custom paging) μπορεί να γίνει σε ένα σημείο, χωρίς αλλαγή σε κάθε interface ξεχωριστά. Συγκεκριμένα:

- **ProjectRepository:** Αποτελεί το σημείο επαφής με τη βάση για τα αντικείμενα των Έργων (Projects). Αναλαμβάνει την καταχώρηση νέων έργων, την ανάκτησή τους για προβολή λεπτομερειών, καθώς και την οριστική διαγραφή τους μαζί με τα συσχετιζόμενα δεδομένα τους.
- **UseCaseRepository:** Διαχειρίζεται την αποθήκευση και ανάκτηση των Περιπτώσεων Χρήσης. Επιτρέπει την ανάκτηση όλων των απαιτήσεων που ανήκουν σε ένα συγκεκριμένο έργο, την ενημέρωση της κατάστασης έγκρισής τους (π.χ. PENDING, APPROVED) και τη διαγραφή τους.
- **CrcCardRepository:** Είναι υπεύθυνο για τη διαχείριση των Καρτών CRC στη βάση δεδομένων. Εξασφαλίζει τη μοναδικότητα του ονόματος της κλάσης (Class Name) ανά έργο κατά τη δημιουργία ή επεξεργασία και υποστηρίζει την ανάκτηση των καρτών για την παραγωγή διαγραμμάτων.
- **ActorRepository:** Αναλαμβάνει την αποθήκευση και αναζήτηση των εμπλεκόμενων χρηστών (Actors) στα διαγράμματα συστήματος, εξυπηρετώντας κυρίως τη σύνδεσή τους με τις Περιπτώσεις Χρήσης και τη συγκέντρωσή τους για την παραγωγή του διαγράμματος Use Case.
- **ParticipantRepository:** Διαχειρίζεται τις εγγραφές συμμετοχής των χρηστών στα έργα. Χρησιμοποιείται κατά τον διαμοιρασμό για να ελέγξει αν ένας χρήστης ανήκει ήδη στο έργο και για να ανακτήσει τον ακριβή ρόλο του (π.χ. Συνεργάτης, Αξιολογητής) ώστε να επιβληθούν τα σωστά δικαιώματα πρόσβασης.
- **UserRepository:** Είναι υπεύθυνο για την επικοινωνία με τον πίνακα των εγγεγραμμένων χρηστών. Αξιοποιείται κατά την αναζήτηση χρήστη με βάση το Username (όπως στην πρόσκληση συμμετεχόντων στο ή την αλλαγή καθολικών ρόλων από τον Διαχειριστή).
- **CommentRepository:** Διαχειρίζεται την αποθήκευση και ανάκτηση των παρατηρήσεων (Σχολίων). Υποστηρίζει τη χρονολογική ανάκτηση σχολίων για μια

συγκεκριμένη Περίπτωση Χρήσης ή CRC Κάρτα, καθώς και την οριστική διαγραφή τους από τον συντάκτη τους.

E. DTOs

Τα DTOs αποτελούν τα ελαφριά αντικείμενα μεταφοράς δεδομένων ανάμεσα στις HTML φόρμες και την επιχειρησιακή λογική. Δεν είναι entities της βάσης δεδομένων, δεν διατηρούν σχέσεις με άλλες οντότητες και δεν περιέχουν business logic· περιορίζονται μόνο στα πεδία που εισάγει ο χρήστης σε μια συγκεκριμένη οθόνη. Έτσι, οι πραγματικές entities του domain παραμένουν απομονωμένες από το επίπεδο παρουσίασης και η εφαρμογή κανόνων εγκυρότητας μπορεί να γίνει με Bean Validation annotations στο ίδιο το σημείο εισόδου.

F. Exceptions (Διαχείριση Σφαλμάτων)

Κάθε επιχειρησιακή κατάσταση σφάλματος αναπαρίσταται από μια ξεχωριστή εξαίρεση τύπου RuntimeException, η οποία μεταφέρει ένα ευανάγνωστο μήνυμα για τον τελικό χρήστη και επιτρέπει στον GlobalExceptionHandler να αποφασίσει για την κατάλληλη απόκριση (flash message, redirect, ή σελίδα 404).

- **ValidationException:** Εγείρεται όταν τα εισερχόμενα δεδομένα μιας φόρμας αποτυγχάνουν τους κανόνες επικύρωσης (π.χ. κενό όνομα έργου, μηδενικοί Actors σε Use Case, διπλότυπο Class Name CRC Card). Είναι το πιο διαδεδομένο exception σε όλα τα services.
- **UnauthorizedException:** Εγείρεται όταν ο τρέχων χρήστης επιχειρεί ενέργεια για την οποία δεν έχει δικαίωμα (π.χ. διαγραφή έργου που δεν του ανήκει, διαγραφή σχολίου άλλου χρήστη). Καθοδηγεί το σύστημα να επιστρέψει σαφές μήνυμα απαγόρευσης.
- **EmptyCommentException:** Ειδική περίπτωση, η οποία τονίζει στον χρήστη ότι δεν επιτρέπεται η υποβολή κενού σχολίου, χωρίς να συγχέεται με τα γενικά validation exceptions.
- **InsufficientDataException:** Εγείρεται από το DiagramService όταν ο χρήστης ζητά παραγωγή διαγράμματος αλλά το έργο δεν περιέχει αρκετά δεδομένα (καθόλου UseCases ή CrcCards). Επιτρέπει στο UI να εμφανίσει στοχευμένο μήνυμα.
- **UserNotFoundException:** Χρησιμοποιείται κατά τη διαδικασία πρόσκλησης συμμετέχοντα (UC12) όταν το username που πληκτρολογήθηκε δεν αντιστοιχεί σε εγγεγραμμένο χρήστη.
- **AlreadyParticipantException:** Προστατεύει την ακεραιότητα του πίνακα Participant όταν επιχειρείται η προσθήκη χρήστη που ήδη συμμετέχει στο έργο.

- **RoleUnchangedException:** Εγείρεται από το `UserService.changeUserRole` όταν ο Διαχειριστής επιχειρεί να αναθέσει ξανά τον τρέχοντα ρόλο σε έναν χρήστη, αποτρέποντας περιττές ενημερώσεις της βάσης.
- **NotFoundException:** Νέα ενοποιημένη εξαίρεση για όλες τις περιπτώσεις "δεν βρέθηκε" (Project, UseCase, CrcCard, Comment, User, Participant). Αντικαθιστά τις διάσπαρτες ad-hoc αναφορές που υπήρχαν στις προηγούμενες εκδόσεις και χαρτογραφείται από τον `GlobalExceptionHandler` σε σελίδα HTTP 404 με σχετικό μήνυμα.

G. Security / Config Layer (Ασφάλεια)

Το επίπεδο ασφάλειας αναμορφώθηκε ώστε να αντικατοπτρίζει πιστά την τελική υλοποίηση. Η εφαρμογή είναι κλασικό server-rendered web application με Thymeleaf views και βασίζεται σε session-based authentication. Το `SecurityConfig` αξιοποιεί τις ενσωματωμένες δυνατότητες του Spring Security για να συνδυάσει `formLogin`, `BCrypt password hashing` και `custom UserDetailsService`.

- **SecurityConfig:** Δηλώνεται ως `@Configuration @EnableWebSecurity` και εκθέτει τέσσερα beans. Ορίζει την πολιτική προσπέλασης: τα endpoints `/login`, `/register` καθώς και τα στατικά αρχεία (`/css/**`, `/js/**`, `/images/**`) είναι ανοιχτά σε όλους, οι σελίδες `/admin/**` περιορίζονται αυστηρά σε χρήστες με `ROLE_ADMIN`, ενώ κάθε άλλο request απαιτεί authentication. Το `formLogin` ρυθμίζεται κατάλληλα από εδώ, ενώ το `logout` ανακατευθύνει σωστά για την έξοδο. Παράλληλα, το CSRF προστατεύει τις HTML φόρμες σε επίπεδο view.
- **passwordEncoder():** Επιστρέφει `BCryptPasswordEncoder`, ώστε όλοι οι κωδικοί να αποθηκεύονται κρυπτογραφημένοι με salted hashing. Χρησιμοποιείται τόσο κατά την εγγραφή όσο και κατά την αυθεντικοποίηση.
- **authenticationProvider():** Δημιουργεί `DaoAuthenticationProvider`, στον οποίο εγχέεται το `UserDetailsService` και ο `PasswordEncoder`. Έτσι, το Spring Security γνωρίζει πώς να φορτώσει τον χρήστη από τη βάση και πώς να συγκρίνει τον υποβληθέντα κωδικό με τον αποθηκευμένο hash.
- **authenticationManager():** Εκθέτει το standard `AuthenticationManager` του Spring, προκειμένου να μπορεί να αξιοποιηθεί από οποιοδήποτε σημείο της εφαρμογής χρειάζεται προγραμματική αυθεντικοποίηση.

H. Strategies

- **UseCaseDiagramStrategy:** Interface με μέθοδο η οποία καθορίζει τον τρόπο παραγωγής script για τα Use Case Diagrams.
- **ClassDiagramStrategy:** Interface με μέθοδο η οποία καθορίζει τον τρόπο παραγωγής script για τα Class Diagrams.
- **PlantUmlUseCaseStrategy:** Υλοποιεί το UseCaseDiagramStrategy και παράγει script συμβατό με τη σύνταξη του PlantUML. Δηλώνει actor lifelines για κάθε Actor, usecase lifelines για κάθε UseCase, και τις συνδέσεις μεταξύ τους.
- **NomnomlUseCaseStrategy:** Υλοποιεί το UseCaseDiagramStrategy αλλά παράγει script σε σύνταξη Nomnoml, για χρήστες που προτιμούν το συγκεκριμένο εργαλείο.
- **PlantUmlClassStrategy:** Υλοποιεί το ClassDiagramStrategy και παράγει PlantUML class diagram script από τις Κάρτες CRC του έργου, αξιοποιώντας τα πεδία className, responsibilities και collaborations.
- **NomnomlClassStrategy:** Υλοποιεί το ClassDiagramStrategy παράγοντας το αντίστοιχο script σε σύνταξη Nomnoml.

Με αυτή την οργάνωση, το DiagramService παραμένει εντελώς ανέπαφο από λεπτομέρειες σύνταξης διαγραμμάτων: του αρκεί να επιλέξει το κατάλληλο strategy και να καλέσει τη μέθοδο build. Η προσθήκη νέου διαγραμματικού εργαλείου (π.χ. Mermaid) θα απαιτούσε μόνο την υλοποίηση δύο νέων κλάσεων που υλοποιούν τα interfaces, χωρίς καμία αλλαγή στον υπάρχοντα κώδικα του service ή των Controllers.

Συνολική Αρχιτεκτονική του Συστήματος

Το SpecFlow στη v1.0 ακολουθεί μια κλασική διαστρωματωμένη αρχιτεκτονική (layered architecture) τύπου Spring Boot MVC, με σαφή διαχωρισμό ευθυνών και μονόδρομη εξάρτηση από πάνω προς τα κάτω. Κάθε αίτημα του χρήστη διασχίζει διαδοχικά τα επίπεδα: ξεκινά από τη σελίδα (view), φιλτράρεται από το επίπεδο ασφάλειας, περνά στον αντίστοιχο Controller, ο οποίος επικαλείται την επιχειρησιακή λογική (Service), και τέλος καταλήγει στη βάση δεδομένων μέσω των Repositories.

Το **UI / HTTP Layer** δέχεται τα αιτήματα του χρήστη και επιστρέφει τις HTML σελίδες. Πάνω από αυτό λειτουργεί διαθεματικά το **Security Layer**, που εξασφαλίζει ότι κάθε αίτημα εκτελείται από ταυτοποιημένο χρήστη με τα κατάλληλα δικαιώματα. Το **Business Logic Layer** συγκεντρώνει την επιχειρησιακή λογική του συστήματος (επικύρωση, εξουσιοδότηση, κανόνες ροής) και αξιοποιεί

το **Strategy Pattern** για ευέλικτη παραγωγή διαγραμμάτων. Το **Data Access Layer**, στηριγμένο στο Spring Data JPA, απομονώνει την πρόσβαση στη βάση δεδομένων από τον υπόλοιπο κώδικα, ώστε η επιχειρησιακή λογική να μη γνωρίζει λεπτομέρειες αποθήκευσης. Στη βάση όλων βρίσκεται το **Domain Layer**, που μοντελοποιεί τις βασικές οντότητες του προβλήματος.

Παράλληλα προς αυτή την κάθετη ροή, υπάρχουν δύο οριζόντια επίπεδα υποστήριξης: το **DTO Layer**, που μεταφέρει δεδομένα από τις φόρμες προς τα services χωρίς να εκθέτει τις πραγματικές entities, και το **Exceptions Layer**, που παρέχει ένα ομοιόμορφο λεξιλόγιο σφαλμάτων το οποίο διαχειρίζεται κεντρικά ο GlobalExceptionHandler.

Συνολικά, η αρχιτεκτονική σέβεται τις θεμελιώδεις αρχές καθαρού κώδικα — διαχωρισμός ευθυνών, εξάρτηση από αφαιρέσεις και όχι από υλοποιήσεις, αρχή ανοιχτό/κλειστό μέσω του Strategy Pattern — και αφήνει το σύστημα ανοιχτό σε μελλοντικές επεκτάσεις (π.χ. πραγματική αποστολή email, νέα διαγραμματικά εργαλεία, REST API επιπέδου) χωρίς αναδιάρθρωση του πυρήνα.

Εργαλεία που χρησιμοποιήθηκαν

Η συγγραφή του παρόντος τεχνικού κειμένου έγινε με την χρήση του Microsoft Word. Το “Domain Model Diagram” υλοποιήθηκε με τη χρήση του εργαλείου plant-uml.